



Day 5 · Git 团队协作 与全栈应用部署

从 Git 小团队协作到宝塔、Docker、域名与 HTTPS

主讲

Tauh

单位

烛龙智元 · DragonAI

日期

Vibe Coding 10 天训练营 · 2026

课程地图

Git 协作

分支、Commit、Push、冲突处理与主分支治理

宝塔部署

服务器、Nginx、前端构建、FastAPI / Node 进程守护

Docker

Dockerfile、Compose、数据库、更新与回滚

公网运维

DNS、域名、反向代理、SSL、日志与故障定位

01

开场与课程主线

Git 协作、服务器部署、域名与 HTTPS

1.1 今天只讲两条主线

Git 团队协作 + 全栈应用部署

◆ 主线 A · Git 协作

- ◆ 用分支隔离任务
- ◆ 用 Commit 记录可解释的变化
- ◆ 用 Push 共享分支，同伴检查后合并
- ◆ 用规则保护主分支

◆ 主线 B · 公网部署

- ◆ 服务器持续运行进程
- ◆ Nginx 接住公网请求
- ◆ 宝塔或 Docker 管理应用
- ◆ 域名 + SSL 提供可信入口

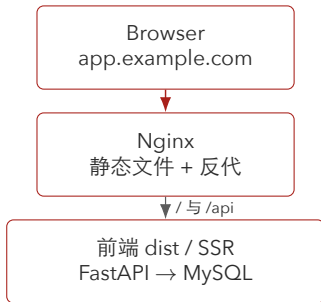
贯穿项目

Node 技术栈前端 + Python FastAPI 后端 + MySQL，可替换为 Node 后端

1.2 演示项目与部署结构

Node 前端 + FastAPI 后端 + MySQL

- ◆ 前端：Vite / React / Vue 等
Node 技术栈
- ◆ 包管理：pnpm +
pnpm-lock.yaml
- ◆ 后端：FastAPI + uv + uv.lock
- ◆ 数据：MySQL，生产环境不暴露
公网端口
- ◆ 入口：/ 访问前端，/api 访问后
端



02

Git 核心模型与本地工作流

用 Commit、Branch 与 Diff 管理代码历史

2.1 Git 的三个区域

工作区、暂存区、仓库



- ◆ `git add` 选择进入下一个 Commit 的变化。
- ◆ 一次 Commit 应该表达一个完整、可解释、可回滚的意图。

2.2 第一次使用 Git 的身份配置

作者信息会写入每个 Commit

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"  
  
git config --global init.defaultBranch main  
git config --global --list
```

- ◆ 公司与个人账号需要区分时，可在项目目录使用不带 **--global** 的配置。
- ◆ 提交邮箱应与 GitHub 账号关联，才能正确显示贡献身份。
- ◆ 认证使用 SSH Key 或受控 Token；密码与 Token 留在受控环境。

2.3 获取仓库：init 与 clone

一个创建新历史，一个复制已有历史

◆ 新项目

```
mkdir customer-service  
cd customer-service  
git init  
git status
```

◆ 已有项目

```
git clone git@github.com:org/app.git  
cd app  
git remote -v
```

团队默认

使用 Clone 获取完整历史与远程仓库配置

2.4 一次最小提交循环：从 git add 开始

选择文件、检查暂存区、创建本地 Commit

```
git add docs/git-demo-notes.md
git diff --cached
git commit -m "docs: add Git workflow demo notes"
```

- ◆ **git add** 选择本次提交包含的文件或代码块。
- ◆ **git diff --cached** 检查即将提交的内容。
- ◆ **git commit** 只写入本机仓库的 .git 历史。

2.5 Commit 写入本地仓库

Commit 0fd229a

```
$ git diff --cached --stat
docs/git-demo-notes.md | 13 ++++++++
1 file changed, 13 insertions(+)

$ git commit -m "docs: add Git workflow demo notes"
[main 0fd229a] docs: add Git workflow demo notes
1 file changed, 13 insertions(+)
create mode 100644 docs/git-demo-notes.md
```

此刻

Commit 已存在于本地，GitHub 仍停留在上一个 Commit

2.6 Push 同步到 GitHub

Commit 是本地动作，Push 是网络动作

◆ Push 前

```
$ git status -sb
## main...origin/main [ahead 1]

$ git log --oneline --decorate -2
0fd229a (HEAD -> main) docs: add
  Git workflow demo notes
8ce893a (origin/main) feat: add
  deployment and operations lessons
```

◆ Push 后

```
$ git push
8ce893a..0fd229a  main -> main

$ git status -sb
## main...origin/main
```

2.7 好 Commit 的三个标准

小、完整、能读懂

标准	好例子	坏例子
单一意图	登录表单接 API	登录 + 首页改版 + 升级依赖
可独立验证	提交后测试能通过	中间状态无法运行
信息明确	<code>fix(api): handle 401</code>	<code>update / 改一下</code>

- ◆ 常用前缀: feat、fix、docs、refactor、test、chore。
- ◆ Commit 是给未来的自己、Review 者和回滚操作看的。

2.8 用 log 与 show 追踪变化

查看提交顺序、指针位置与文件变化

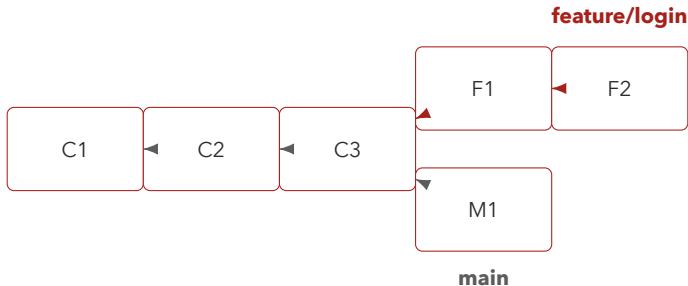
```
$ git log --oneline --decorate --graph -4
* 0fd229a (HEAD -> main, origin/main)
  docs: add Git workflow demo notes
* 8ce893a feat: add deployment and operations lessons
* bef37be feat: add Git collaboration lessons
* ab2afa2 chore: initialize Day 5 beamer project

$ git show --stat 0fd229a
docs/git-demo-notes.md | 13 ++++++++
1 file changed, 13 insertions(+)
```

- ◆ HEAD -> main, origin/main 表示本地与 GitHub 已对齐。
- ◆ `git show 0fd229a` 查看某次提交的说明与完整 Diff。
- ◆ `git diff ab2afa2..0fd229a` 比较两个历史节点。

2.9 分支是什么

指向某个 Commit 的可移动标签



- ◆ 分支是指向 Commit 的轻量指针。
- ◆ 每个任务独立分支，可以并行开发、单独 Review、随时放弃。

2.10 分支的日常命令

从最新 main 开始工作

```
git switch main
git pull --ff-only

git switch -c feature/login-api
# edit, test, commit...

git switch main
git branch -d feature/login-api
```

- ◆ 分支名建议：feature/login-api、fix/chat-scroll、docs/deploy-guide。
- ◆ `--ff-only` 避免 `git pull` 意外生成本地合并提交。
- ◆ 分支合并后及时删除，避免团队继续在过期分支上工作。

2.11 撤销操作：先判断“是否已经共享”

公开历史优先 revert，未共享历史才考虑重写

场景	推荐命令	影响
未暂存的文件改坏	<code>git restore file</code>	丢弃本地变化
误加入暂存区	<code>git restore --staged file</code>	保留文件变化
最近 Commit 信息错	<code>git commit --amend</code>	重写未共享提交
已推送的错误 Commit	<code>git revert <commit></code>	新增反向提交，历史安全

红线

共享分支不执行 `reset --hard` 或普通 `push --force`

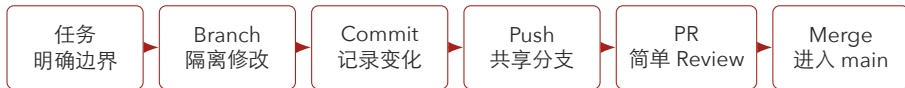
03

GitHub 小团队协作流程

任务分支、小步提交、同步 main、合并检查

3.1 小团队开发的完整流程

一个任务、一个分支、一个合并目标



- ◆ 任务先明确负责人、修改范围与完成条件。
- ◆ 每个任务使用独立分支，Commit 保持小而清晰。
- ◆ 推送到在线仓库后，由同伴快速检查并合并到 main。

3.2 开发者的日常命令流

同步 main、创建分支、提交、推送

```
git switch main
git pull --ff-only
git switch -c feature/chat-history

# edit and test
git add src/
git diff --cached
git commit -m "feat(chat): load conversation history"

git push -u origin feature/chat-history
```

下一步

在 GitHub 发起合并请求，由一名同伴快速检查后合并

3.3 合并前快速检查

围绕本次任务检查逻辑、测试、配置与密钥

检查项	小团队做法
改动范围	确认只包含当前任务，排除无关格式化与临时文件
核心逻辑	检查错误处理、接口变化和关键业务路径
测试结果	运行构建、测试或最小可行验证，记录检查结果
配置安全	排除 .env、密码、密钥和本地专用配置
合并收尾	检查通过后进入 main，删除已完成的功能分支

控制范围

一次合并只解决一个问题，降低检查与回滚成本

3.4 开发期间同步 main

小团队优先使用 Merge，规则保持统一

```
git fetch origin
git switch feature/chat-history
git merge origin/main

# resolve conflicts, test, commit
git push
```

- ◆ 功能分支开发超过半天时主动同步 main。
- ◆ 冲突在自己的分支解决并测试，避免把问题留到最终合并。
- ◆ 团队熟悉历史重写后，再统一讨论是否使用 Rebase。

3.5 密钥必须留在 Git 之外

泄漏后立即轮换并清理历史

严禁提交

- .env、数据库密码、JWT Secret、云厂商密钥、模型 API Key、私钥文件。
- ◆ 发现泄漏后立即吊销或轮换密钥，再处理仓库文件。
- ◆ 清理 Git 历史、Fork、缓存和 CI 日志，并检查泄漏原因。
- ◆ 部署时通过服务器环境变量、Secret 文件或平台 Secret 注入。

提交前检查

`git diff --cached` + Secret Scanner + 最小权限密钥

04

冲突处理、Review 与主分支治理

读标记、解冲突、保护 main

4.1 先学会读冲突标记

当前分支、分隔线、待合并分支

```
<<<<<<< HEAD
const apiBase = "/api";
=====
const apiBase = "http://localhost:8000";
>>>>>>> feature/api-config
```

- ◆ 先确认生产架构与配置约定，再选择或融合两侧代码。
- ◆ 最终结果可能选择一边，也可能融合成第三种写法。
- ◆ 删除全部标记后，必须重新运行测试与构建。

4.2 命令行解决冲突的标准步骤

定位、编辑、验证、继续

```
git status
# open conflicted files and resolve markers

pnpm lint
pnpm test
pnpm build

git add src/config/api.ts
git commit # merge workflow
# or: git rebase --continue
```

- ◆ `git status` 会列出所有未合并路径。
- ◆ 不确定时可用 `git merge --abort` 或 `git rebase --abort` 回到操作前。
- ◆ 解决冲突时必须理解两边业务意图。

4.3 GitHub 在线解决简单行冲突

复杂冲突回到本地测试



- ◆ 简单的竞争行可以使用网页冲突编辑器。
- ◆ 删除 / 重命名、生成文件、依赖锁文件等复杂冲突建议本地处理。
- ◆ 合并前必须验证构建、测试和关键业务流程。

来源: [GitHub Docs · Resolving a merge conflict](#)

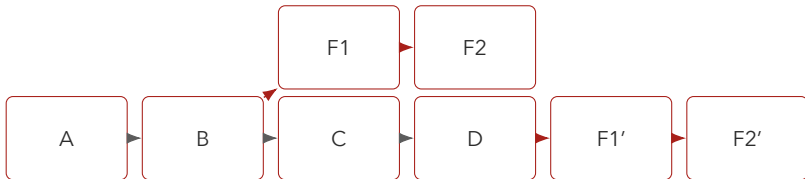
4.4 如何减少冲突

重点是工作拆分、同步和接口约定

做法	原理
小任务、短生命周期分支	缩短多人修改的重叠时间与范围
明确模块负责人	减少多人同时改同一核心文件
频繁同步 main	尽早暴露竞争变化
格式化单独提交	避免真实逻辑变化被噪声淹没
先沟通公共接口	Schema、路由、配置文件先约定再编码
锁文件由实际安装命令生成	不手工编辑 <code>pnpm-lock.yaml</code> / <code>uv.lock</code>

4.5 Rebase 为什么需要谨慎

它会把 Commit 复制到新的父节点上



- ◆ Rebase 后 F1/F2 变成新的 F1'/F2'，Commit ID 改变。
- ◆ 只重写自己的功能分支；共享历史保持稳定。

4.6 Rebase 后推送用 force-with-lease

它会检查远程分支是否被别人更新

```
git fetch origin  
git rebase origin/main  
  
git push --force-with-lease
```

禁止使用

`git push --force` 可能覆盖同事刚推送的 Commit。

边界

如果团队尚未理解历史重写，统一用 Merge 同步 main 更安全

4.7 主分支保护规则

main 始终代表 “可部署版本”

- ◆ 禁止直接 Push，所有变化通过受控合并流程进入 main。
- ◆ 至少一人 Approve，关键目录可配置 CODEOWNERS。
- ◆ 必须通过构建、测试、Lint 等状态检查。
- ◆ 必须解决 Review 对话后才能合并。
- ◆ 默认禁止 Force Push 与删除主分支。

结果

规则把 “团队口头约定” 变成平台自动执行的门禁

来源: [GitHub Docs](#) · [About protected branches](#)

05

服务器购买与部署架构

服务器配置、端口与部署路线

5.1 购买服务器

选择地域、系统、配置与网络

地域

靠近用户；境内上线关注备案

系统

Ubuntu / Debian LTS 纯净镜像

配置

课堂建议 2C4G 起步

网络

公网 IPv4、带宽、22 / 80 / 443

◆ 购买后记录

- ◆ 公网 IPv4、SSH 用户与密钥
- ◆ 实例名称与到期时间
- ◆ 云防火墙规则
- ◆ 控制台重置与救援入口

提醒：境内上线应依法判断备案或许可要求。

来源：工信部·非经营性互联网信息服务备案管理办法

5.2 端口开放策略

公网保留 22、80、443

端口	典型用途	建议
22	SSH	仅管理使用；可限制来源 IP、使用密钥登录
80	HTTP	证书验证与跳转 HTTPS
443	HTTPS	用户正式访问入口
3000	Node / Next.js	默认仅本机或容器网络
8000	FastAPI	默认仅本机或容器网络
3306	MySQL	不向公网开放，应用走本机 / 内网

来源：阿里云轻量应用服务器 · 防火墙

5.3 宝塔与 Docker 的分工

可视化运维与可重复交付

维度	宝塔部署	Docker 部署
学习门槛	面板可视化，上手快	需要理解镜像、网络、卷
环境一致性	依赖服务器当前状态	镜像描述运行环境
迁移复现	需要记录手工步骤	Dockerfile + Compose 可重复
排错入口	面板日志 + 系统日志	docker logs + 容器状态
适用	新手、小型单机项目	团队交付、多服务、持续迭代

组合方式

宝塔管理 Nginx 与证书，Docker Compose 运行应用

06

通过宝塔部署全栈项目

用面板管理 Nginx、进程、数据库与证书

6.1 安装宝塔后的第一件事：加固面板

面板是高权限管理入口

1. 从宝塔官网复制当前系统对应的官方安装命令。
2. 保存安装输出中的面板地址、初始用户名和密码。
3. 首次登录立即修改强密码、面板端口与安全入口。
4. 云防火墙限制面板端口来源，优先绑定固定管理 IP。
5. 开启面板 HTTPS、登录告警与必要的双因素验证。

供应链提醒

不使用“破解版 / 纯净版 / 开心版”等第三方修改包，只从官方渠道安装与更新。

来源：宝塔官方下载

6.2 安装基础环境

Nginx、Node、pnpm、Python 与 MySQL

组件	用途	选择建议
Nginx	80 / 443、静态文件、反向代理	必装
Node.js	构建前端、运行 Node 后端 / SSR	安装项目要求的 LTS 版本
pnpm	Node 依赖与构建	通过 Corepack 或官方方式启用
Python + uv	FastAPI 运行环境	与 pyproject.toml 要求一致
MySQL	业务数据	本机部署时仅监听本地 / 内网

原则：项目版本来自仓库声明，生产升级要有计划。

6.3 服务器获取代码的推荐方式

Git Clone + 只读 Deploy Key

```
cd /www/wwwroot
git clone git@github.com:your-org/customer-service.git
cd customer-service

git switch main
git pull --ff-only
git rev-parse --short HEAD
```

- ◆ 服务器只负责部署时，Deploy Key 尽量设为只读。
- ◆ 记录当前 Commit ID，后续更新与回滚都以它为坐标。
- ◆ 不在服务器直接改业务代码，否则 Git 历史与线上状态会分叉。

6.4 构建 Node 前端

依赖锁定，API 地址使用生产配置

```
cd /www/wwwroot/customer-service/frontend  
  
corepack enable  
pnpm install --frozen-lockfile  
pnpm build  
  
ls -la dist
```

- ◆ Vite 常见输出目录是 `dist/`；其他框架以项目配置为准。
- ◆ 推荐前端请求相对路径 `/api`，由 Nginx 转发到后端。
- ◆ 构建时环境变量会写入静态文件，修改后必须重新构建。

6.5 创建静态站点

网站根目录指向构建产物

1. 宝塔左侧进入“网站”，添加站点并填写域名。
2. 站点类型选择纯静态；根目录指向前端 `dist/`。
3. 确认运行用户对目录有读取权限。
4. 暂时可用 HTTP 验证首页资源是否正确加载。
5. 浏览器 F12 检查 HTML、JS、CSS 是否全部 200。

网站根目录

只指向 `dist/` 等生产构建产物

来源：宝塔官方文档 · 添加站点

6.6 SPA 刷新 404 怎么修

让未知前端路由回到 index.html

```
location / {  
    try_files $uri $uri/ /index.html;  
}
```

- ◆ React Router / Vue Router 的 History 模式由浏览器管理路由。
- ◆ 用户直接访问 /chat/123 时，Nginx 找不到同名文件会返回 404。
- ◆ **try_files** 让前端路由接管；/api 保持独立代理规则。

6.7 准备 FastAPI 生产环境

服务器按锁文件创建独立虚拟环境

```
cd /www/wwwroot/customer-service/backend  
  
uv sync --locked --no-dev  
uv run python -m pytest  
  
uv run fastapi run app/main.py \  
    --host 127.0.0.1 --port 8000 --proxy-headers
```

- ◆ 先前台启动并用 `curl http://127.0.0.1:8000/health` 验证。
- ◆ 监听 127.0.0.1，业务端口只允许本机 Nginx 访问。
- ◆ 确认无误后再交给宝塔项目管理 / 应用管理器守护。

6.8 FastAPI 进程守护

SSH 断开后仍运行，异常后能重启

配置项	示例
项目目录	/www/wwwroot/customer-service/backend
Python 环境	项目 .venv 中的 Python / FastAPI
启动命令	使用上一页命令，监听 127.0.0.1:8000
运行用户	非 root，确保能读代码与写日志 / 上传目录
环境变量	通过面板项目配置或受控 Env 文件注入
重启策略	开机启动、异常退出自动重启

版本差异

不同宝塔版本入口可能显示为 Python 项目、应用管理器或进程守护

6.9 Nginx 把 /api 转给 FastAPI

同域部署减少 CORS 与 Cookie 问题

```
location /api/ {  
    proxy_pass http://127.0.0.1:8000/;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}
```

- ◆ 访问 `https://app.example.com/api/health`, Nginx 转给 FastAPI。
- ◆ 注意 `proxy_pass` 末尾斜杠会影响路径改写, 必须现场验证真实路由。
- ◆ 后端启用 Proxy Headers 后, 才能正确感知外部协议与客户端信息。

来源: [NGINX Docs](#) · [Reverse Proxy](#)

6.10 宝塔反向代理界面

面板字段与 Nginx 配置一一对应

- ◆ 代理名称: fastapi-api
- ◆ 代理目录: /api/
- ◆ 目标 URL:
http://127.0.0.1:8000
- ◆ 发送域名: 通常使用 \$host
- ◆ API 默认关闭缓存

添加反向代理

开启代理 ☒

开启缓存 ☐

高级功能 ☐

* 代理名称

* 目标URL

* 发送域名

内容替换

删除

+ 添加内容替换

- 代理目录: 访问这个目录时将会把目标URL的内容返回并显示(需要开启高级功能)
- 目标URL: 可以填写你需要代理的站点, 目标URL必须为可正常访问的URL, 否则将返回错误
- 发送域名: 将域名添加到请求头传递到代理服务器, 默认为目标URL域名, 若设置不当可能导致代理无法正常运行
- 内容替换: 只能在使用nginx时提供, 最多可以添加3条替换内容, 如果不需要替换请留空

取消

确定

来源: [宝塔官方文档](#) · 反向代理

6.11 前后端联调的验证顺序

后端、Nginx、浏览器逐层检查

1. `curl http://127.0.0.1:8000/health`: 后端进程是否正常。
2. `curl -i http://127.0.0.1/api/health`: Nginx 反代是否正常。
3. 浏览器打开域名: 前端静态资源是否正常。
4. F12 Network 查看 `/api/...` 请求、状态码、响应体。
5. 后端日志确认同一请求是否到达、耗时与错误。

定位法

在哪一层第一次失败，问题就位于该层或它的下一跳

6.12 Node.js 后端部署

pnpm 构建 + PM2 / 宝塔 Node 项目守护

```
cd /www/wwwroot/customer-service/server
```

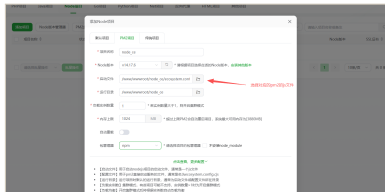
```
corepack enable  
pnpm install --frozen-lockfile  
pnpm build  
pnpm start
```

- ◆ Express / NestJS 等后端通常监听 127.0.0.1:3000。
- ◆ 宝塔 Node 项目管理默认可使用 PM2 做守护与自动重启。
- ◆ Nginx 的目标 URL 改为 http://127.0.0.1:3000，其他思路相同。

来源：宝塔官方文档 · Node.js PM2 部署

6.13 宝塔 Node 项目配置示意

项目路径、启动命令、端口和运行用户



添加 Node 项目 · 宝塔官方



配置启动端口 · 宝塔官方

pnpm

找不到命令时先确认 Node 版本与 PATH，避免重复安装

来源：宝塔官方文档 · Next.js 部署

6.14 环境变量如何放到服务器

仓库保存变量名，服务器保存真实值

类型	处理方式
前端公开配置	构建时注入；默认视为用户可见，不放 Secret
后端普通配置	项目环境变量或权限受控的 Env 文件
数据库密码 / API Key	最小权限、定期轮换、日志脱敏
JWT Secret	使用足够随机值，变更会影响现有 Token
上传目录	独立持久目录，权限给运行用户并纳入备份

权限

Env 文件只允许部署用户 / 运行用户读取，避免通过静态站点暴露

6.15 宝塔手工更新顺序

构建、备份、切换、验证

```
cd /www/wwwroot/customer-service
git fetch origin
git switch main
git pull --ff-only

cd frontend
pnpm install --frozen-lockfile
pnpm build

cd ../backend
uv sync --locked --no-dev
# run migrations, then restart backend in panel
```

- ◆ 更新前记录旧 Commit ID、备份数据库与上传文件。
- ◆ 依赖、迁移、重启、健康检查必须按 SOP 执行。
- ◆ 不在用户高峰期第一次尝试未知升级步骤。

6.16 宝塔部署最常见的 502

Nginx 找不到或无法连接上游服务

- ◆ 后端进程没有启动或持续崩溃。
- ◆ 代理目标端口写错。
- ◆ 应用只监听容器 / 其他网卡。
- ◆ 权限、依赖或环境变量导致启动失败。
- ◆ Nginx 配置未重载或语法错误。

◆ 排查证据

- ◆ 项目状态与启动日志
- ◆ `ss -lntp`
- ◆ 本机 `curl` 上游端口
- ◆ Nginx error log
- ◆ 应用 error / access log

顺序

先验证上游进程，再检查 Nginx 配置与日志

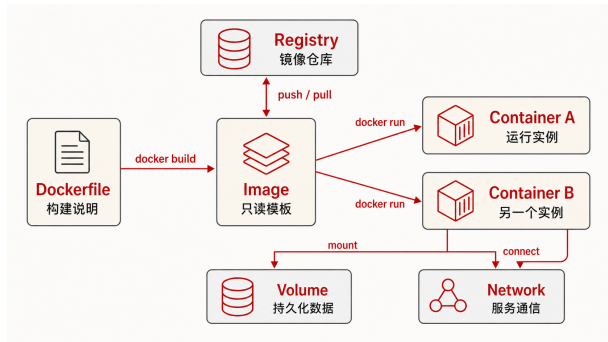
07

Docker 基础与部署模型

镜像、容器、网络、卷与 Compose

7.1 Docker 核心部件的关系

Dockerfile、Image、Container、Registry、Volume 与 Network



同一 Image 可启动多个 Container；Volume 保存数据，Network 负责容器通信。

7.2 Claude Code: 准备 Docker 部署

一段可直接复制的专业提示词

你是资深 DevOps 工程师。请先扫描当前仓库，再为生产服务器准备 Docker 部署。

已知技术栈：

- 前端：Node.js + pnpm；判断是静态构建还是 Next.js / Node SSR。
- 后端：Python FastAPI + uv，必须遵循 pyproject.toml 与 uv.lock。
- 数据库：MySQL；公网入口由 Nginx 提供，同域 /api 转发后端。

请按顺序完成：

1. 检查目录、构建脚本、启动命令、端口、环境变量、迁移、健康检查和上传目录。
2. 先输出部署拓扑、假设、风险与将要新增/修改的文件，然后直接实现。
3. 为每个服务创建生产级 Dockerfile：多阶段构建、锁文件安装、最小运行镜像。
4. 创建 .dockerignore，排除 .git、.env、依赖、缓存、构建物和本地虚拟环境。
5. 创建 compose.yaml：frontend、api、db、healthcheck、restart、内部 network、MySQL named volume；只把网关绑定到宿主机 127.0.0.1。
6. 创建 Nginx 配置：SPA fallback 或 SSR 反代、/api、代理头、WebSocket / SSE。
7. 创建 .env.example，只保留变量名和安全示例，禁止写入任何真实 Secret。
8. 提供数据库迁移、启动、日志、健康检查、更新、备份与回滚命令。

约束：

- 不猜测缺失的业务信息；会改变架构时先提出问题。
- 保留现有可用配置，修改前说明原因；不要提交 .env、密码或密钥。
- 执行 docker compose config，并检查构建命令与服务间连接。
- 最后汇总文件路径、运行步骤、验证方法、风险和待确认项。

7.3 最小 Dockerfile 读法

每条指令都在描述镜像构建过程

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "-m", "uvicorn", "app.main:app",
    "--host", "0.0.0.0", "--port", "8000"]
```

- ◆ **RUN** 在构建时执行；**CMD** 在容器启动时执行。
- ◆ 容器内服务监听 0.0.0.0，才能从容器网络访问。
- ◆ JSON Exec Form 能更正确地接收停止信号。

7.4 Docker 层与缓存

把变化少的步骤放前面

1. 复制依赖清单与锁文件。
2. 安装依赖，生成可缓存层。
3. 再复制变化频繁的业务源码。
4. 执行构建，最后只把运行产物带进最终镜像。

快

依赖不变时复用缓存

小

多阶段构建不携带开发工具

常见错误

先 COPY . . 再安装依赖，任何源码变化都会让依赖层缓存失效

7.5 .dockerignore 控制构建上下文

缩小构建上下文并排除敏感文件

```
.git
.env
node_modules
.pnpm-store
dist
.next
.venv
__pycache__
*.log
coverage
```

- ◆ Docker 构建只应该接收生成镜像所需的文件。
- ◆ 即使 Dockerfile 没有 COPY 某文件，发送巨大构建上下文仍会拖慢构建。
- ◆ .env、私钥和本地虚拟环境必须排除。

7.6 Build、Run、Port 三个动作

左边是宿主机端口，右边是容器端口

```
docker build -t customer-api:1.0 .
```

```
docker run -d \  
  --name customer-api \  
  -p 127.0.0.1:8000:8000 \  
  --restart unless-stopped \  
  customer-api:1.0
```

- ◆ 127.0.0.1:8000:8000 只绑定宿主机本地，供 Nginx 反代。
- ◆ 如果写 8000:8000，通常会绑定全部网卡，应再由防火墙限制。
- ◆ Image Tag 是版本坐标；生产环境使用明确版本或 Commit SHA。

7.7 容器排错的四个命令

状态、日志、进程内检查、配置

```
docker ps -a  
docker logs --tail 200 -f customer-api  
docker exec -it customer-api sh  
docker inspect customer-api
```

- ◆ `docker ps -a` 看退出状态和重启循环。
- ◆ `docker logs` 看应用标准输出与错误输出。
- ◆ `docker exec` 用于诊断，不应在容器内手工修改生产代码。

7.8 Volume 解决数据持久化

容器可替换，数据必须保留

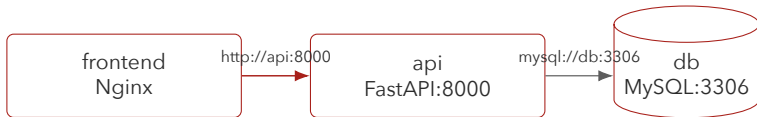
数据	是否放镜像	推荐位置
应用源码 / 构建物	是	Image 只读层
MySQL 数据目录	否	Named Volume / 云数据库
用户上传文件	否	Volume / 对象存储
日志	通常否	标准输出 + 日志平台 / 受控目录
Secret	否	环境变量 / Secret 文件

设计原则

应用容器应尽量无状态，方便重建、替换和横向扩展

7.9 Docker Network 让服务用名字互访

Compose 会为项目创建默认网络



- ◆ 容器间不使用 localhost; localhost 只代表当前容器自己。
- ◆ 使用服务名 api、db 作为网络主机名。
- ◆ 只有需要给宿主机 / 公网访问的服务才映射 ports。

7.10 Healthcheck 与 restart

分别负责健康判断与重启策略

机制	判断对象	用途
restart policy	主进程是否退出	崩溃或重启服务器后恢复容器
healthcheck	应用是否能响应	标记 unhealthy、辅助依赖与监控
readiness	是否可接流量	大型编排平台控制流量进入

- ◆ 健康检查应快速、稳定，不执行昂贵业务。
- ◆ 可检查应用事件循环、关键依赖连接，但避免产生副作用。

7.11 Docker Compose 管理多容器应用

一份 YAML 描述服务、网络、卷与配置

```
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down
```

- ◆ **config** 先检查合并后的配置和变量替换。
- ◆ **down** 默认不删 Named Volume；加 **-v** 会删除数据卷，必须谨慎。
- ◆ Compose 适合单机部署，也是今天全栈 Docker 路线的主角。

来源：Docker Docs · Compose

08

Docker 部署 Node 前端与 FastAPI

Dockerfile 管单个服务，Compose 管整套应用

8.1 静态前端 Dockerfile: 构建阶段

pnpm 安装依赖并生成 dist

```
FROM node:24-slim AS build

ENV PNPM_HOME="/pnpm"
ENV PATH="$PNPM_HOME:$PATH"
RUN corepack enable

WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN --mount=type=cache,id=pnpm,target=/pnpm/store \
    pnpm install --frozen-lockfile

COPY . .
RUN pnpm build
```

缓存关键

先复制 package 与 lockfile; 源码变化不应让依赖安装层全部失效

来源: [pnpm Docs](#) · [Working with Docker](#)

8.2 静态前端 Dockerfile: 运行阶段

运行镜像只包含 Nginx、配置与 dist

```
FROM nginx:alpine AS runtime

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

- ◆ 多阶段构建把开发依赖留在 build 阶段。
- ◆ 运行镜像只包含 Nginx、静态产物与配置，体积更小、攻击面更少。
- ◆ 前端的 Secret 不应在任何阶段注入；浏览器最终都能看到静态配置。

8.3 前端容器内的 Nginx 配置

负责 SPA 与容器网络内 /api 转发

```
server {  
    listen 80;  
    root /usr/share/nginx/html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://api:8000/;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

网络名

api 是 Compose 服务名，由 Docker DNS 在内部解析

8.4 FastAPI Dockerfile: uv + 锁文件

构建可重复的 Python 环境

```
FROM python:3.12-slim

COPY --from=ghcr.io/astral-sh/uv:0.11.28 \
    /uv /uvx /bin/
WORKDIR /app

COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-dev --no-install-project

COPY . .
RUN uv sync --locked --no-dev

ENV PATH="/app/.venv/bin:$PATH"
CMD ["fastapi", "run", "app/main.py",
    "--host", "0.0.0.0", "--port", "8000",
    "--proxy-headers"]
```

来源: [uv Docs](#) · [Using uv with FastAPI](#)

8.5 Compose 第一部分：数据库与 API

变量从服务器 Env 文件注入

```
services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL",
        "mysqladmin ping -h 127.0.0.1 -uroot -p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10
```

8.6 Compose 第二部分：API 与前端

前端映射到宿主主机本地端口

```
api:
  build: ./backend
  env_file: .env.production
  depends_on:
    db:
      condition: service_healthy
  restart: unless-stopped

frontend:
  build: ./frontend
  ports:
    - "127.0.0.1:8080:80"
  depends_on:
    - api
  restart: unless-stopped

volumes:
  mysql_data:
```

暴露面

API 与 MySQL 没有 ports，只能从 Compose 网络访问

8.7 第一次启动的顺序

校验配置、构建、迁移、启动、验证

```
docker compose config
docker compose build

docker compose up -d db
docker compose run --rm api \
  uv run alembic upgrade head

docker compose up -d
docker compose ps
docker compose logs --tail 100
```

- ◆ 迁移工具以项目实际使用为准；核心是“只执行一次、失败就停止发布”。
- ◆ 首次启动后从宿主机测试 `curl http://127.0.0.1:8080/api/health`。

8.8 depends_on 与健康检查

启动顺序、健康状态与业务可用是三件事

- ◆ 普通 `depends_on` 只控制容器启动顺序。
- ◆ 数据库容器启动后，MySQL 仍需要时间初始化。
- ◆ `Healthcheck + condition: service_healthy` 能减少早期连接失败。
- ◆ 应用自身仍应对短暂依赖故障实现有限重试与清晰日志。

故障处理

密码、迁移或网络错误应修复根因，避免无限重启

8.9 宿主机 Nginx 接入 Compose

宝塔继续负责域名与证书

```
location / {  
    proxy_pass http://127.0.0.1:8080;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}
```

- ◆ 宝塔 Nginx 终止 HTTPS，再把整站请求转给前端容器。
- ◆ 前端容器内部 Nginx 再把 /api 路由到 api:8000。
- ◆ 两层代理便于课堂理解；生产也可把职责合并到一层 Nginx。

8.10 Next.js / Node SSR

运行 Node Server, 由 Nginx 反向代理

```
# next.config.ts
const nextConfig = { output: "standalone" };

# runtime stage
FROM node:24-slim AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV HOSTNAME=0.0.0.0
ENV PORT=3000
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
CMD ["node", "server.js"]
```

来源: [Docker Docs · Containerize a Next.js application](#)

8.11 Node 后端与 FastAPI 的 Compose 差异

替换运行镜像与启动命令

项目	FastAPI	Node.js API
依赖锁	<code>uv.lock</code>	<code>pnpm-lock.yaml</code>
构建	<code>uv sync --locked</code>	<code>pnpm install --frozen-lockfile</code>
启动	<code>fastapi run</code>	<code>node dist/main.js / pnpm start</code>
容器端口	8000	3000
代理目标	<code>api:8000</code>	<code>api:3000</code>

架构复用

域名、Nginx、数据库、Secret、日志、更新与回滚逻辑保持一致

8.12 pnpm Monorepo 部署

只把目标应用与生产依赖带进镜像

- ◆ 多包仓库可以在 build 阶段完成全量构建。
- ◆ 使用 `pnpm deploy --filter=<app> --prod <target>` 生成自包含部署目录。
- ◆ 每个服务从目标目录创建独立运行镜像。
- ◆ pnpm 11 默认要求 Workspace 正确配置 injected packages；升级前读版本文档。

价值

避免把整个 Monorepo、其他服务源码与全部开发依赖复制到每个镜像

来源: [pnpm Docs](#) · [pnpm deploy](#)

8.13 Docker 更新流程

重建镜像，原地替换容器

```
git fetch origin
git switch main
git pull --ff-only

docker compose build
docker compose run --rm api \
  uv run alembic upgrade head
docker compose up -d

docker compose ps
docker compose logs --tail 100
```

生产升级

更成熟的流程由 CI 构建带 Commit SHA 的不可变镜像，服务器只负责 pull 与切换

8.14 Docker 回滚范围

镜像、数据库、上传文件与配置

对象	回滚策略
应用镜像	保留上一个稳定 Tag，Compose 切回旧 Tag
数据库 Schema	优先向后兼容迁移；破坏性迁移前做备份与演练
用户上传	独立 Volume / 对象存储，不随容器替换
环境变量	版本化记录变量名与变更，不提交真实值
Nginx / Compose	配置进入 Git，变更前校验语法

最高风险

新代码写入了旧版本无法理解的数据时，单纯切回旧镜像可能仍然失败

09

域名、反向代理与 SSL 证书

域名找服务器，Nginx 分流，证书建立信任

9.1 A 记录与 CNAME 记录

一个指向 IP，一个指向另一个主机名

类型	示例	适用
A	@ → 203.0.113.10	根域名指向 IPv4 服务器
AAAA	@ → IPv6	使用 IPv6 时
CNAME	www → example.com	子域名别名或第三方服务
TXT	验证字符串	域名验证、邮件安全、ACME DNS-01

- ◆ @ 通常代表根域名，www 代表 `www.example.com`。
- ◆ 修改记录后存在缓存与 TTL，不同递归 DNS 生效时间可能不同。

来源: [Cloudflare DNS Docs](#) · [Record examples](#)

9.2 用 dig 验证 DNS

同时检查 Host Header 与站点匹配

```
dig +short app.example.com
dig app.example.com A
nslookup app.example.com

curl -I http://203.0.113.10 \
-H "Host: app.example.com"
```

- ◆ **dig** 验证域名解析结果。
- ◆ 带 Host Header 访问 IP，可区分 DNS 问题与 Nginx 站点配置问题。
- ◆ 使用 CDN / 代理 DNS 时，查询结果通常指向代理节点。

9.3 一个域名如何承载前后端

推荐同域 + 路径分流

方案	示例	特点
同域路径	app.example.com/ /api	+ Cookie、CORS 最简单，课程默认
不同子域	app.example.com api.example.com	+ 边界清晰，需要配置 CORS / Cookie
业务端口	example.com:8000	不适合作为正式用户入口

统一入口：用户只访问 443，应用真实端口留在内部。

9.4 代理头传递外部请求信息

协议、域名与客户端 IP

```
proxy_set_header Host $host;  
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
proxy_set_header X-Forwarded-Proto $scheme;
```

- ◆ 后端生成重定向 URL、记录审计日志时需要外部请求信息。
- ◆ FastAPI 位于可信代理后时使用 `--proxy-headers`。
- ◆ 只信任可信反向代理写入的 X-Forwarded-For。

来源: [FastAPI Docs](#) · [Behind a Proxy](#)

9.5 WebSocket 与 SSE 代理配置

AI 流式输出的超时与缓冲

```
# WebSocket
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";

# Server-Sent Events / streaming
proxy_buffering off;
proxy_read_timeout 3600s;
```

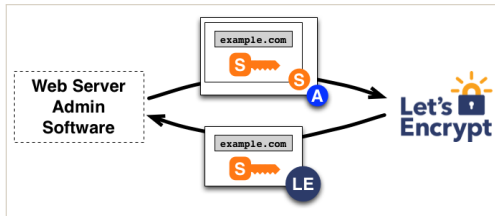
- ◆ 普通 JSON 接口可以使用默认缓冲；流式响应需要尽快向客户端转发。
- ◆ 超时过短会让长回答中途断开；过长也要配合连接与资源限制。
- ◆ 在浏览器 Network 中确认数据持续到达。

9.6 Let's Encrypt / ACME 自动签发

验证域名控制权并定期续期

1. ACME Client 请求验证挑战。
2. 通过 HTTP 文件或 DNS 记录证明控制域名。
3. CA 从网络侧验证挑战。
4. 生成 CSR 并签发证书。
5. 客户端自动安装并定期续期。

来源: [Let's Encrypt · How It Works](#)



9.7 HTTP-01 与 DNS-01 怎么选

课堂单域名优先 HTTP-01

验证方式	要求	适用
HTTP-01	域名已指向服务器，80 端口可访问	单域名、普通网站，配置简单
DNS-01	能通过 DNS 添加 TXT 记录 / API	泛域名证书、80 不可达、集中证书管理

- ◆ HTTP-01 申请失败先查 DNS、80 端口、防火墙和站点匹配。
- ◆ DNS API Token 使用最小权限，并像生产密钥一样保护。

9.8 宝塔申请与部署 SSL

站点设置中完成申请、安装与续签



来源：宝塔官方文档 · 部署 SSL 证书

- ◆ 域名先正确解析到服务器。
- ◆ 进入站点 SSL 页面申请。
- ◆ 可选免费证书或上传已有证书。
- ◆ 开启强制 HTTPS 与到期提醒。

9.9 HTTP 跳转 HTTPS

80 端口保留验证与重定向

```
server {  
    listen 80;  
    server_name app.example.com;  
  
    location /.well-known/acme-challenge/ {  
        root /www/server/panel/vhost/certbot;  
    }  
  
    location / {  
        return 301 https://$host$request_uri;  
    }  
}
```

- ◆ 宝塔通常会自动生成对应配置，不必手写路径。
- ◆ 强制 HTTPS 后，前端 API、图片、字体和 WebSocket 都应使用安全协议。

9.10 Mixed Content 排错

HTTPS 页面中的 HTTP 请求会被拦截

典型错误

页面是 `https://app.example.com`，前端仍请求
`http://203.0.113.10:8000/api`。

- ◆ 浏览器会阻止不安全的 Fetch、XHR、Script、Stylesheet 等请求。
- ◆ 让 API、脚本、图片与 WebSocket 全部使用安全协议。
- ◆ 使用相对地址 `/api`，可同时避免协议、域名与环境差异。

来源：MDN · Mixed content

9.11 CORS 配置

浏览器对跨 Origin 请求的约束

场景	处理
同域 /api	同源请求，配置最简
app.example.com → api.example.com	后端明确允许前端 Origin
带 Cookie / Authorization Postman / curl 能通，浏览器不通	不能用任意 *；需精确 Origin 与 Credentials 配置 优先检查 CORS 与 Preflight

安全边界

CORS 不能替代后端鉴权；攻击者可以不用浏览器直接请求 API

9.12 上线后验证域名与证书

检查跳转、证书、接口和安全头

```
curl -I http://app.example.com
curl -I https://app.example.com
curl -i https://app.example.com/api/health

openssl s_client \
  -connect app.example.com:443 \
  -servername app.example.com </dev/null 2>/dev/null |
openssl x509 -noout -subject -issuer -dates
```

- ◆ HTTP 应跳转 HTTPS；证书域名与有效期正确；API 返回预期 JSON。
- ◆ 浏览器控制台检查 Mixed Content、CORS、证书链与资源 404。

9.13 证书续签与 HSTS

自动续签、到期提醒与安全策略

- ◆ 免费证书通常周期较短，依赖自动续签；开启到期提醒。
- ◆ 续签失败常见原因：DNS 已切换、80 被关闭、验证路径被代理、DNS Token 失效。
- ◆ 确认 HTTPS 长期稳定后再考虑 HSTS。
- ◆ HSTS 配错会让浏览器拒绝回退 HTTP，证书错误时也无法绕过。

运维要求

持续监控自动续签与到期时间

来源：MDN · [Strict-Transport-Security](#)

10

运维、排错、回滚与课堂实战

日志、排错、备份、更新与恢复

10.1 线上故障按层定位

从用户入口一路追到数据层



- ◆ 每层都用独立工具验证，找到“第一次失败”的位置。
- ◆ 页面报 500 时，从首次失败的层级开始处理。

10.2 日志分层看什么

同一请求最好能用 Request ID 串起来

日志层	关注字段	常见问题
Nginx Access	Host、Path、Status、耗时	请求是否到达、慢在哪
Nginx Error	upstream、connect、time-out	502、证书、配置
应用日志	Request ID、用户、异常栈	业务错误、第三方 API
Docker / 进程	Exit Code、Restart Count	启动失败、崩溃循环
MySQL	连接、慢查询、磁盘	密码、迁移、性能

日志纪律

不记录密码、Token、完整身份证件与模型密钥；生产错误要可行动

10.3 502 Bad Gateway 决策树

Nginx 到上游的连接失败

```
# 1. Is the process/container running?  
docker compose ps  
docker compose logs --tail 200 api  
  
# 2. Is the port listening?  
ss -lntp | grep 8000  
  
# 3. Can the host/proxy reach it?  
curl -i http://127.0.0.1:8000/health  
  
# 4. Is Nginx config valid?  
nginx -t
```

结论

本机 curl 都失败，先修应用；本机成功但域名失败，再查 Nginx 与防火墙

10.4 404 有三种完全不同的来源

先看响应内容和 Server Header

现象	可能来源	修复方向
刷新前端子路由 404	Nginx 找不到文件	SPA <code>try_files</code>
/api 返回前端 HTML	路由顺序 / 回退误吞 API	独立 location /api/
FastAPI JSON 404	路径被代理改写错误	检查尾斜杠与 root path
静态 JS / CSS 404	构建 base path 错	检查资源前缀与站点根目录

10.5 数据库连接失败怎么分层

网络、账号、Schema、迁移

1. 主机名是否正确：容器内用 db，宿主机进程可能用 127.0.0.1。
2. 端口是否正确，数据库是否真正 Ready。
3. 用户、密码、数据库名是否与初始化配置一致。
4. 用户是否有目标数据库权限，字符集是否符合项目要求。
5. 当前代码需要的迁移是否已执行。

容器陷阱

API 容器里的 localhost:3306 指向 API 容器自身

10.6 服务器资源不足的症状

构建时和运行时的瓶颈不同

```
free -h
df -h
du -sh /var/lib/docker/*
docker stats
top
```

- ◆ Node 前端构建常见内存峰值；MySQL、模型服务与多 Worker 会长期占内存。
- ◆ 磁盘可能被 Docker Image、Build Cache、日志和数据库共同吃满。
- ◆ OOM Kill 时应用像“突然消失”，需要查系统日志和容器退出码。

10.7 备份范围与恢复验证

数据库、上传文件、配置与证书

对象	备份方式	恢复验证
MySQL	定时逻辑备份 + 云快照	新库导入并跑关键查询
上传文件	Volume / 目录同步 / 对象存储版本	随机抽样文件可打开
Compose / Nginx Env 变量名	进入 Git <code>.env.example</code> + Secret 清单	新服务器可复现 新环境变量齐全
证书	可自动重新签发为主	续签任务与域名验证可用

3-2-1

重要数据至少 3 份、2 种介质、1 份异地；课堂项目也要养成习惯

10.8 单机部署的安全基线

默认拒绝，按需开放

- ◆ SSH 使用密钥，限制 root 登录
- ◆ 面板入口限制来源并启用 HTTPS
- ◆ 公网只开放 22 / 80 / 443
- ◆ MySQL、Redis、API 端口不暴露公网
- ◆ 运行进程使用非 root 用户
- ◆ 系统与依赖定期更新
- ◆ Secret 最小权限并定期轮换
- ◆ 数据库与上传文件定期备份
- ◆ 日志与磁盘空间设告警
- ◆ 删除离职成员与过期 Deploy Key

现实边界

一台服务器适合小型项目和教学；业务增长后再拆数据库、对象存储与负载均衡

10.9 一次规范发布包含哪些阶段

发布前、中、后都要有证据

阶段	动作
准备	分支已合并、测试通过、记录版本、备份、确认回滚点
构建	锁文件安装、生产构建、镜像打 Tag、扫描敏感信息
迁移	执行兼容性数据库迁移，失败则停止
发布	重启 / 替换进程或容器，保持变更范围最小
验证	Health、首页、登录、数据库、关键 API、日志
观察	错误率、响应时间、资源、用户反馈
收尾	记录结果，删除临时权限，更新变更日志

10.10 回滚触发条件

提前定义阈值与决策人

- ◆ 健康检查持续失败，服务无法启动。
- ◆ 登录、支付、核心对话等关键链路不可用。
- ◆ 错误率或响应时间明显超过基线。
- ◆ 数据写入异常、权限越权、Secret 泄漏等安全事件。
- ◆ 修复时间不可控，继续在线排查会扩大影响。

决策原则

先恢复服务，再分析根因；回滚本身也需要验证与记录

10.11 从手工部署走向 CI/CD

今天先把手工 SOP 做对，再自动化



- ◆ 自动化把已验证的 SOP 固化为可重复流程。
- ◆ CI 使用与本地一致的 pnpm / uv 版本与锁文件策略。

10.12 课堂综合实战

把 Day 3 / Day 4 项目真正交付给另一个人访问

1. 确认上线前修复已经合并到主分支。
2. 购买服务器后，完成宝塔基础环境配置。
3. 选择宝塔手工路线或 Docker Compose 路线部署。
4. 配置域名、/api 反向代理与 SSL。
5. 另一组使用手机网络访问并完成关键流程测试。
6. 主动制造一次错误端口，按日志定位并恢复。
7. 更新一个版本，再回滚到上一个版本。

10.13 官方参考资料 · 协作与运行时

- ◆ [Pro Git Book](#) · [Git 基础与分支](#)
- ◆ [GitHub Docs](#) · [GitHub Flow](#)
- ◆ [GitHub Docs](#) · [About Pull Requests](#)
- ◆ [GitHub Docs](#) · [Protected Branches](#)
- ◆ [pnpm Docs](#) · [Working with Docker](#)
- ◆ [uv Docs](#) · [Using uv in Docker](#)
- ◆ [FastAPI Docs](#) · [FastAPI in Containers](#)
- ◆ [Docker Docs](#) · [Use Compose in Production](#)

10.14 官方参考资料 · 宝塔、网络与 HTTPS

- ◆ [宝塔面板官方文档](#)
- ◆ [宝塔 · 反向代理](#)
- ◆ [宝塔 · 部署 SSL 证书](#)
- ◆ [宝塔 · Node.js PM2 部署](#)
- ◆ [NGINX Docs · Reverse Proxy](#)
- ◆ [Cloudflare Docs · DNS](#)
- ◆ [Let's Encrypt · How It Works](#)
- ◆ [MDN · Mixed Content](#)
- ◆ [工业和信息化部 · 互联网信息服务备案相关规定](#)

交付完成

稳定交付，持续运行